

Pflichtmodul Informationssysteme (SoSe 2025)

Prof. Dr. Jens Teubner

Leitung der Übungen: Stephanie Althoff, Jannik Wolff

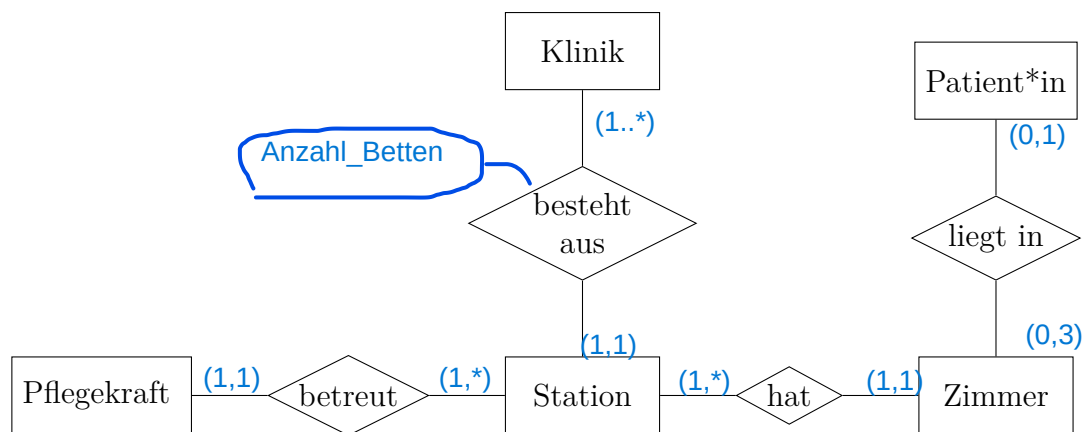
Übungsblatt Nr. 4

Ausgabe: 28.04.2025

Abgabe: 05.05.2025 – 23:59 Uhr

Aufgabe 1 (Entity-Relationship Diagramm)

Gegeben ist das folgende ER-Diagramm zusammen mit einer Anforderungsanalyse, die einen Teil eines Krankenhaussystems darstellen:



Anforderungsanalyse: Die Klinik ist in verschiedene Stationen unterteilt, wobei jede Station mit mehreren Zimmern ausgestattet ist und von mehreren Pflegekräften betreut wird. Jede Pflegekraft ist dabei ausschließlich einer Station zugeordnet. Patientinnen und Patienten erhalten entweder ambulante oder stationäre Behandlungen. Ein belegtes Zimmer ist entweder mit einer Patientin/einem Patienten oder mit drei Patient*innen belegt (es handelt sich um Einzel- oder Dreibettzimmer). Es ist zudem vorgeschrieben, dass in jedem Zimmer nur Patientinnen und Patienten des gleichen Geschlechts untergebracht werden dürfen.

Aufgaben:

1. Überlegen Sie sinnvolle Kardinalitäten für die Beziehungen und geben Sie diese in der (min, max)-Notation an.
2. Gibt es in der obigen Beschreibung Integritätsbedingungen, die nicht durch Kardinali-

tätsangaben modelliert werden können?

eine Mögliche integritätsbedingungen: in einem Zimmer dürfen entweder Männer oder Frauen untergebracht werden, könnte noch hinzugefügt werden.

3. Ergänzen Sie die Entitäten im ER-Diagramm mit jeweils drei sinnvollen Attributen. Unterstreichen Sie dabei das Schlüsselattribut. **Klinik: KID(schlüssel), Name. Ort**
4. Ergänzen Sie eine der Beziehungen im ER-Diagramm mit einem sinnvollen Attribut.
Das Verb
5. Formulieren Sie einen SQL-Befehl (in der CREATE TABLE-Syntax) zur Erstellung der Tabelle Klinik. Sie können den Befehl in DuckDB ausprobieren.

```
create Table Klinik(  
KID  INTEGER NOT NULL,  
Name CHAR(30),  
Ort  CHAR(30),  
PRIMARY KEY (KID))  
;
```

Aufgabe 2 (Logische Datenunabhängigkeit)

In Moodle steht Ihnen die Datei

`pres_social_media.sql`

zur Verfügung. Diese enthält SQL-Befehle zum erstellen einer neuen Tabelle PRES_SOCIAL_MEDIA mit fiktiven Posts einiger Präsidenten.

Aufgaben:

1. Führen Sie die SQL-Befehle in Ihrer lokalen DuckDB-Instanz mit der in Blatt 02 erstellten Präsidentendatenbank aus. Die Tabelle umfasst Spalten wie PRES_NAME, PLATFORM, CONTENT und POST_DATE. Um die gesamte Struktur der Tabelle einzusehen, verwenden Sie den Befehl: DESCRIBE PRES_SOCIAL_MEDIA;
2. Erstellen Sie eine View PRES_TWEETS, die alle Posts der Plattform "Twitter" zeigt. Die View soll den Namen des Präsidenten, dessen Partei, den Inhalt des Posts und das Er-

```
CREATE VIEW PRES_TWEETS as  
SELECT pres_name,content, post_date  
from pres_social_media;  
where platform='twitter';
```

stellungsdatum enthalten. Nutzen Sie dazu die `CREATE VIEW`-Syntax¹.

```
create view pres_tweets as
select pres_name, content, post_date
from pres_social_media
where platform = 'twitter';
```

Nachdem Sie die View in DuckDB erstellt haben, sollten Sie mit

```
SELECT * FROM PRES_TWEETS;
```

alle entsprechenden Tweets anzeigen können.

3. Es kommt vor, dass bestimmte Posts wegen ihres Inhalts gelöscht werden müssen. Bisher wurden diese Daten dauerhaft aus der Tabelle entfernt. Zukünftig sollen Posts erhalten bleiben und nur als *gelöscht* markiert werden. Nutzer*innen der Datenbank sollen von dieser konzeptionellen Änderung nichts bemerken.

In Moodle finden Sie die Datei

```
pres_social_media_removed.sql
```

Diese enthält SQL-Befehle um der Tabelle `PRES_SOCIAL_MEDIA` eine Spalte `REMOVE_DATE` hinzuzufügen. Zudem werden einige gelöschte Posts eingefügt. Ein Post wird als *gelöscht* betrachtet, wenn das `REMOVE_DATE` nicht `NULL` ist.

- (a) Führen Sie alle Befehle aus der Datei `pres_social_media_removed.sql` in Ihrer lokalen DuckDB-Instanz aus.
- (b) Geben Sie alle Posts über die View aus. Sie werden nun neu hinzugekommene Posts sehen, die eigentlich als *gelöscht* markiert wurden. `select * from Pres_tweets;`
- (c) Ändern Sie die View² `PRES_TWEETS`, so dass gelöschte Posts nicht mehr angezeigt werden. Nach der Änderung sollte die Abfrage `SELECT * FROM PRES_TWEETS;` wieder das ursprüngliche Ergebnis liefern, ohne die als gelöscht markierten Posts angezeigt werden. Geben Sie die SQL-Anweisung zur Änderung der View an.

```
create view pres_tweets as
select pres_name, content, post_date
from pres_social_media
where remove_date is NULL
```

¹https://duckdb.org/docs/sql/statements/create_view.html

²Verwenden Sie dafür die `CREATE OR REPLACE VIEW`-Syntax.